LABSHEET-5

Q1. Student Encapsulation [Easy] .

Create a class Student with private variables name, rollNo, and marks

• Create getter and setter methods for all variables

• Create display Details().

• Create one Student object.

• Set values through setters and display values through getters. Concepts: private, getter, setter, data hiding.

```
class Student {
private String name;
private int rollNo;
private double marks;

public void setName(String name) {
  this.name = name;
}

public String getName() {
  return name;
}

public void setRollNo(int rollNo) {
  this.rollNo = rollNo;
}

public int getRollNo() {
  return rollNo;
}

public void setMarks(double marks) {
```

```
    this.marks = marks;
  }

  public double getMarks() {
    return marks;
  }

  public void displayDetails() {
    System.out.println("Name: " + getName());
    System.out.println("Roll No: " + getRollNo());
    System.out.println("Marks: " + getMarks());
  }

  public static void main(String[] args) {
    Student s = new Student();

    s.setName("Arpit Kumar");
    s.setRollNo(101);
    s.setMarks(85.5);

    s.displayDetails();
  }
}
```

OUTPUT:-

```
Name: Arpit Kumar
Roll No: 101
Marks: 85.5
```

Q2. Bank Account with Validation [Easy → Medium]

Create a BankAccount class with private accountNumber, accountHolder, and balance.

• Create getters and setters.

• Create deposit(double amount).

• Create withdraw(double amount).

• Deposit must be greater than 0

• Withdrawal must not exceed the current balance.

• Display appropriate messages for invalid transactions. Concepts: encapsulation, controlled access, validation.

```
class BankAccount {
   private int accountNumber;
   private String accountHolder;
   private double balance;

   public void setAccountNumber(int accountNumber) {
      this.accountNumber = accountNumber;
   }

   public int getAccountNumber() {
      return accountNumber;
   }

   public void setAccountHolder(String accountHolder) {
      this.accountHolder = accountHolder;
   }

   public String getAccountHolder() {
      return accountHolder;
   }

   public void setBalance(double balance) {
      this.balance = balance;
   }
```

```
public double getBalance() {
    return balance;
}

public void deposit(double amount) {
    if (amount > 0) {
        balance = balance + amount;
        System.out.println("Amount deposited: " + amount);
    } else {
        System.out.println("Invalid deposit amount.");
    }
}

public void withdraw(double amount) {
    if (amount <= 0) {
        System.out.println("Invalid withdrawal amount.");
    } else if (amount > balance) {
        System.out.println("Insufficient balance.");
    } else {
        balance = balance - amount;
        System.out.println("Amount withdrawn: " + amount);
    }
}

public void displayDetails() {
    System.out.println("Account Number: " + getAccountNumber());
    System.out.println("Account Holder: " + getAccountHolder());
    System.out.println("Balance: " + getBalance());
}
```

```
    public static void main(String[] args) {
        BankAccount b = new BankAccount();

        b.setAccountNumber(101);
        b.setAccountHolder("Arpit Kumar");
        b.setBalance(5000);

        b.deposit(2000);
        b.withdraw(1500);
        b.withdraw(7000);

        b.displayDetails();
    }
}
```

OUTPUT:-

```
Amount deposited: 2000.0
Amount withdrawn: 1500.0
Insufficient balance.
Account Number: 101
Account Holder: Arpit Kumar
Balance: 5500.0
```

Q3. Employee Salary Encapsulation [Medium]

Create an Employee class with private employeeId, employeeName, and salary.

- Create getters and setters.
- Salary cannot be negative.
- Salary greater than 1,000,000 must also be rejected
- Display employee details.
- Test at least one valid and one invalid salary. Concept: validation inside a setter.

```
class Employee {
```

```
private int employeeId;
private String employeeName;
private double salary;

public int getEmployeeId() {
   return employeeId;
}

public void setEmployeeId(int employeeId) {
   this.employeeId = employeeId;
}

public String getEmployeeName() {
   return employeeName;
}

public void setEmployeeName(String employeeName) {
   this.employeeName = employeeName;
}

public double getSalary() {
   return salary;
}

public void setSalary(double salary) {
   if (salary < 0 || salary > 1000000) {
      System.out.println("Invalid salary. Salary must be between 0 and 1000000.");
   } else {
      this.salary = salary;
   }
```

```
    }

    public void displayDetails() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Employee Name: " + employeeName);
        System.out.println("Salary: " + salary);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e1 = new Employee();

        e1.setEmployeeId(101);
        e1.setEmployeeName("Arpit Kumar");
        e1.setSalary(500000);

        System.out.println("Employee Details:");
        e1.displayDetails();

        System.out.println("\nTesting Invalid Salary:");
        e1.setSalary(1500000);
    }
}
```

OUTPUT:-

```
Employee Details:
Employee ID: 101
Employee Name: Arpit Kumar
Salary: 500000.0
```

Testing Invalid Salary:

Invalid salary. Salary must be between 0 and 1000000.

Q4. Animal → Dog [Easy]

Create a parent class Animal containing eat() and sleep().

Create a child class Dog containing bark().

• Use extends. • Create a Dog object.

• Call all three methods.

Draw: Animal ↓ Dog Concept: Single inheritance.

```
class Animal {
   void eat() {
      System.out.println("Animal is eating.");
   }

   void sleep() {
      System.out.println("Animal is sleeping.");
   }
}

class Dog extends Animal {
   void bark() {
      System.out.println("Dog is barking.");
   }
}

public class Main {
   public static void main(String[] args) {
      Dog d = new Dog();
```

```
    d.eat();
    d.sleep();
    d.bark();
  }
}
```

OUTPUT:-

Animal is eating.

Animal is sleeping.

Dog is barking.

Inheritance diagram:

Animal

↓

Dog

Q5. Person → Student [Easy → Medium]

Create Person with name, age, and displayPerson().

Create Student extends Person and add rollNo,

course, and displayStudent(). Create a Student object and display all information. Concepts: single inheritance and code reuse.

```
class Person {
  String name;
  int age;

  void displayPerson() {
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
  }
}
```

```
class Student extends Person {
   int rollNo;
   String course;

   void displayStudent() {
      System.out.println("Roll No: " + rollNo);
      System.out.println("Course: " + course);
   }
}

public class Main {
   public static void main(String[] args) {
      Student s = new Student();

      s.name = "Arpit Kumar";
      s.age = 18;
      s.rollNo = 101;
      s.course = "BCA";

      s.displayPerson();
      s.displayStudent();
   }
}
```

OUTPUT:-

```
Name: Arpit Kumar
Age: 18
Roll No: 101
```

Course: BCA

Inheritance diagram:

Person

↓

Student

Q6. Employee → Manager with Encapsulation [Medium]

Create an encapsulated Employee class with private name and salary and their getters/setters.

Create Manager extends Employee and add department and displayManager().

Use inherited getter/setter methods to access Employee information.

Do not make Employee fields public. Concepts: encapsulation + inheritance.

```
class Employee {
   private String name;
   private double salary;

   public String getName() {
      return name;
   }

   public void setName(String name) {
      this.name = name;
   }

   public double getSalary() {
      return salary;
   }

   public void setSalary(double salary) {
      this.salary = salary;
   }
```

```
}

class Manager extends Employee {
   String department;

   void displayManager() {
      System.out.println("Name: " + getName());
      System.out.println("Salary: " + getSalary());
      System.out.println("Department: " + department);
   }
}

public class Main {
   public static void main(String[] args) {
      Manager m = new Manager();

      m.setName("Arpit Kumar");
      m.setSalary(600000);
      m.department = "IT";

      m.displayManager();
   }
}
```

OUTPUT:-

Name: Arpit Kumar

Salary: 600000.0

Department: IT

Inheritance diagram:

Employee

↓

Manager

Q7. Animal → Dog → Puppy [Easy]

Create three classes:

Animal → eat()

Dog extends Animal → bark()

Puppy extends Dog → play()

Create a Puppy object and call all three methods. Draw the three-level hierarchy.

Concept: multilevel inheritance.

```
class Animal {
   void eat() {
      System.out.println("Animal is eating");
   }
}

class Dog extends Animal {
   void bark() {
      System.out.println("Dog is barking");
   }
}

class Puppy extends Dog {
   void play() {
      System.out.println("Puppy is playing");
   }
}

public class Main {
   public static void main(String[] args) {
      Puppy p = new Puppy();
```

```
    p.eat();
    p.bark();
    p.play();
  }
}
```

OUTPUT:-

Animal is eating

Dog is barking

Puppy is playing

Three-Level Hierarchy

Animal

↓

Dog

↓

Puppy

Q8. Person → Employee → Manager [Medium]

Create:

Person → name, displayName()

Employee extends Person → employeeId, displayEmployee()

Manager extends Employee → department, displayManager()

Create a Manager object and display all information.

Task: identify which member belongs to each class.

Concept: multilevel inheritance.

```
class Person {
  String name = "Arpit Kumar";

  void displayName() {
    System.out.println("Name: " + name);
  }
}
```

```
class Employee extends Person {
   int employeeId = 101;

   void displayEmployee() {
      System.out.println("Employee ID: " + employeeId);
   }
}

class Manager extends Employee {
   String department = "IT";

   void displayManager() {
      System.out.println("Department: " + department);
   }
}

public class Main {
   public static void main(String[] args) {
      Manager m = new Manager();

      m.displayName();
      m.displayEmployee();
      m.displayManager();
   }
}
```

OUTPUT:-

```
Name: Arpit Kumar
Employee ID: 101
Department: IT
```

Three-Level Hierarchy

Person

↓

Employee

↓

Manager

Q9. Vehicle → Car → ElectricCar [Medium]

Create:

Vehicle → start(), stop()

Car extends Vehicle → drive()

ElectricCar extends Car → chargeBattery()

Create an ElectricCar object and demonstrate access to methods from all three levels.

Concepts: multilevel inheritance and inherited behavior.

```
class Vehicle {
  void start() {
    System.out.println("Vehicle is starting");
  }

  void stop() {
    System.out.println("Vehicle is stopping");
  }
}

class Car extends Vehicle {
  void drive() {
    System.out.println("Car is driving");
  }
}

class ElectricCar extends Car {
```

```
  void chargeBattery() {
    System.out.println("Electric car battery is charging");
  }
}

public class Main {
  public static void main(String[] args) {
    ElectricCar e = new ElectricCar();

    e.start();
    e.stop();
    e.drive();
    e.chargeBattery();
  }
}
```

OUTPUT:-

```
Vehicle is starting
Vehicle is stopping
Car is driving
Electric car battery is charging
```

Three-Level Hierarchy:-

```
    Vehicle
    /    \
  start()  stop()
     |
     ↓
    Car
     |
   drive()
     |
```

↓

ElectricCar

|

chargeBattery()

Q10. Animal → Dog and Cat [Easy]

Create: Animal → eat()

Dog extends Animal → bark()

Cat extends Animal → meow()

Create objects of Dog and Cat.

Demonstrate that both can use eat().

Draw the hierarchy.

Concept: hierarchical inheritance.

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Cat is meowing");
    }
}
```

```
public class Main {
   public static void main(String[] args) {
      Dog d = new Dog();
      Cat c = new Cat();

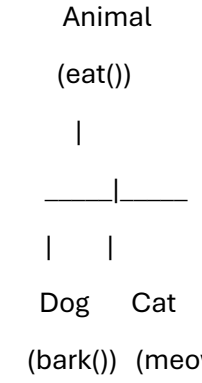
      d.eat();
      d.bark();

      c.eat();
      c.meow();
   }
}
```

OUTPUT:-

```
Animal is eating
Dog is barking
Animal is eating
Cat is meowing
```

Hierarchy Diagram:-



Q11. Person → Student and Teacher [Medium]

Create Person with name and displayName().

Create Student extends Person with course and study().

Create Teacher extends Person with subject and teach().

Create one Student and one Teacher object and display their details. Concept: hierarchical inheritance.

```
class Person {
   String name;

   void displayName() {
      System.out.println("Name: " + name);
   }
}

class Student extends Person {
   String course;

   void study() {
      System.out.println("Course: " + course);
      System.out.println("Student is studying");
   }
}

class Teacher extends Person {
   String subject;

   void teach() {
      System.out.println("Subject: " + subject);
      System.out.println("Teacher is teaching");
   }
}

public class Main {
   public static void main(String[] args) {
```

```
        Student s = new Student();
        s.name = "Rahul";
        s.course = "BCA";

        Teacher t = new Teacher();
        t.name = "Priya";
        t.subject = "Java";

        System.out.println("Student Details:");
        s.displayName();
        s.study();

        System.out.println();

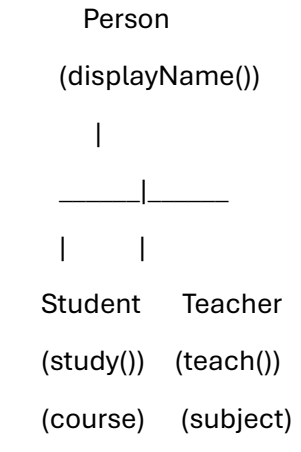
        System.out.println("Teacher Details:");
        t.displayName();
        t.teach();
    }
}
```

OUTPUT:-

```
Student Details:
Name: Rahul
Course: BCA
Student is studying

Teacher Details:
Name: Priya
Subject: Java
Teacher is teaching
```

Hierarchy Diagram:-



Q12. Employee → Developer and Manager [Medium]

Create Employee with employee name, employee ID, and displayEmployee().

Create Developer extends Employee with programmingLanguage and writeCode().

Create Manager extends Employee with department and conductMeeting().

Create both child objects. Identify inherited and child-specific members.

Concept: hierarchical inheritance.

```
class Employee {
   String employeeName;
   int employeeID;

   void displayEmployee() {
      System.out.println("Employee Name: " + employeeName);
      System.out.println("Employee ID: " + employeeID);
   }
}

class Developer extends Employee {
   String programmingLanguage;

   void writeCode() {
      System.out.println("Programming Language: " + programmingLanguage);
      System.out.println("Developer is writing code");
```

```
    }
}

class Manager extends Employee {
    String department;

    void conductMeeting() {
        System.out.println("Department: " + department);
        System.out.println("Manager is conducting a meeting");
    }
}

public class Main {
    public static void main(String[] args) {
        Developer d = new Developer();
        d.employeeName = "Rahul";
        d.employeeID = 101;
        d.programmingLanguage = "Java";

        Manager m = new Manager();
        m.employeeName = "Priya";
        m.employeeID = 102;
        m.department = "HR";

        System.out.println("Developer Details:");
        d.displayEmployee();
        d.writeCode();

        System.out.println();
```

```
        System.out.println("Manager Details:");
        m.displayEmployee();
        m.conductMeeting();
    }
}
```

OUTPUT:-

Developer Details:

Employee Name: Rahul

Employee ID: 101

Programming Language: Java

Developer is writing code

Manager Details:

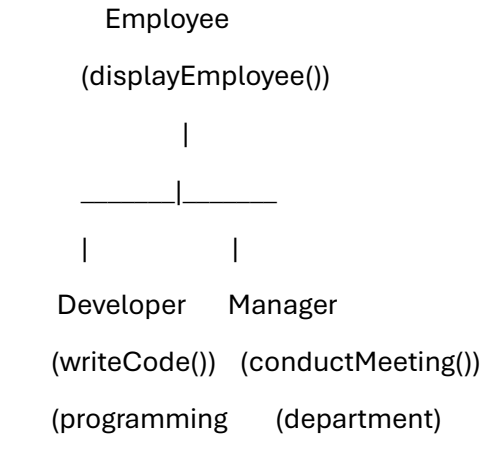
Employee Name: Priya

Employee ID: 102

Department: HR

Manager is conducting a meeting

Hierarchy Diagram:-



Q13. Animal Sound Overriding [Easy → Medium]

Create Animal with makeSound().

Create Dog extends Animal and Cat extends Animal.

Override makeSound() in both child classes.

Dog should produce a barking message and Cat should produce a meowing message.

Use @Override. Concept: method overriding.

```
class Animal {
   void makeSound() {
      System.out.println("Animal makes a sound");
   }
}

class Dog extends Animal {
   @Override
   void makeSound() {
      System.out.println("Dog is barking");
   }
}

class Cat extends Animal {
   @Override
   void makeSound() {
      System.out.println("Cat is meowing");
   }
}

public class Main {
   public static void main(String[] args) {
      Dog d = new Dog();
      Cat c = new Cat();

      d.makeSound();
```

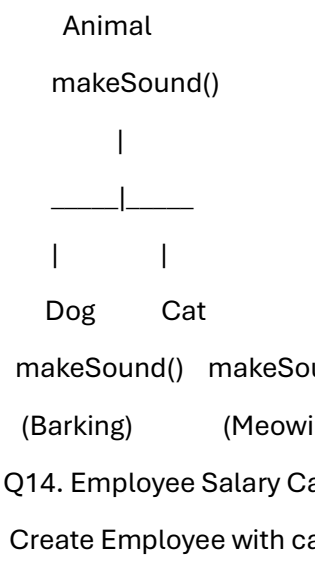
```
        c.makeSound();
    }
}
```

OUTPUT:-

Dog is barking

Cat is meowing

Hierarchy Diagram:-



Q14. Employee Salary Calculation [Medium]

Create Employee with calculateSalary().

Create Manager extends Employee.

Override calculateSalary() in Manager using a different salary calculation.

Create suitable objects and demonstrate which implementation executes.

Concepts: inheritance + method overriding.

```
class Employee {
    double calculateSalary() {
        return 30000;
    }
}


class Manager extends Employee {
    @Override
    double calculateSalary() {
```

```
        return 50000;
    }
}

public class Main {
    public static void main(String[] args) {
        Employee e = new Employee();
        Manager m = new Manager();

        System.out.println("Employee Salary: " + e.calculateSalary());
        System.out.println("Manager Salary: " + m.calculateSalary());
    }
}
```

OUTPUT:-

```
Employee Salary: 30000.0
Manager Salary: 50000.0
```

Hierarchy Diagram:-

```
        Employee
     calculateSalary()
       ₹30,000
          |
        Manager
     calculateSalary()
       ₹50,000
```

Q15. Shape Area Overriding [Medium]

Create Shape with calculateArea().

Create Circle and Rectangle as child classes.

Override calculateArea() in both.

Use appropriate formulas and display the areas.

Use @Override. Concept: same method name with different child implementations.

```
class Shape {
   double calculateArea() {
      return 0;
   }
}

class Circle extends Shape {
   double radius = 5;

   @Override
   double calculateArea() {
      return 3.14 * radius * radius;
   }
}

class Rectangle extends Shape {
   double length = 10;
   double breadth = 5;

   @Override
   double calculateArea() {
      return length * breadth;
   }
}

public class Main {
   public static void main(String[] args) {
      Circle c = new Circle();
      Rectangle r = new Rectangle();
```

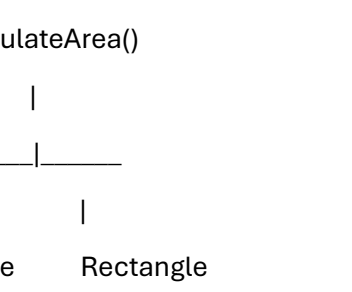
```
        System.out.println("Area of Circle: " + c.calculateArea());
        System.out.println("Area of Rectangle: " + r.calculateArea());
    }
}
```

OUTPUT:-

Area of Circle: 78.5

Area of Rectangle: 50.0

Hierarchy Diagram:-



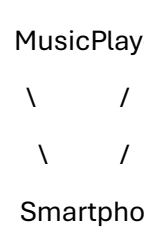
Q16. Camera + MusicPlayer → Smartphone [Medium]

Create interfaces Camera and MusicPlayer.

Camera: takePhoto()

MusicPlayer: playMusic() Create Smartphone implements Camera, MusicPlayer and implement both methods.

Draw:



Concept: multiple inheritance using interfaces.

```
interface Camera {
    void takePhoto();
```

```
}

interface MusicPlayer {
   void playMusic();
}

class Smartphone implements Camera, MusicPlayer {
   @Override
   public void takePhoto() {
      System.out.println("Smartphone is taking a photo");
   }

   @Override
   public void playMusic() {
      System.out.println("Smartphone is playing music");
   }
}

public class Main {
   public static void main(String[] args) {
      Smartphone s = new Smartphone();

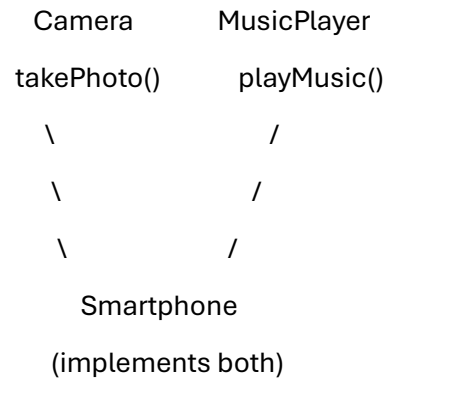
      s.takePhoto();
      s.playMusic();
   }
}
```

OUTPUT:-

Smartphone is taking a photo

Smartphone is playing music

Hierarchy Diagram:-



Q17. Printer + Scanner → MultiFunctionMachine [Medium]

Create interfaces Printer and Scanner.

Printer: print()

Scanner: scan() Create MultiFunctionMachine implementing both.

Also use encapsulation for private machineName and machineId with getters/setters. Demonstrate both interface methods and encapsulated data.

Concepts: multiple interfaces + encapsulation.

```
interface Printer {
   void print();
}

interface Scanner {
   void scan();
}

class MultiFunctionMachine implements Printer, Scanner {
   private String machineName;
   private int machineId;

   public void setMachineName(String name) {
      machineName = name;
   }
```

```
    public String getMachineName() {
        return machineName;
    }

    public void setMachineId(int id) {
        machineId = id;
    }

    public int getMachineId() {
        return machineId;
    }

    @Override
    public void print() {
        System.out.println("Machine is printing");
    }

    @Override
    public void scan() {
        System.out.println("Machine is scanning");
    }
}

public class Main {
    public static void main(String[] args) {
        MultiFunctionMachine m = new MultiFunctionMachine();

        m.setMachineName("HP Printer");
        m.setMachineId(101);
```

```
        System.out.println("Machine Name: " + m.getMachineName());
        System.out.println("Machine ID: " + m.getMachineId());

        m.print();
        m.scan();
    }
}
```

OUTPUT:-

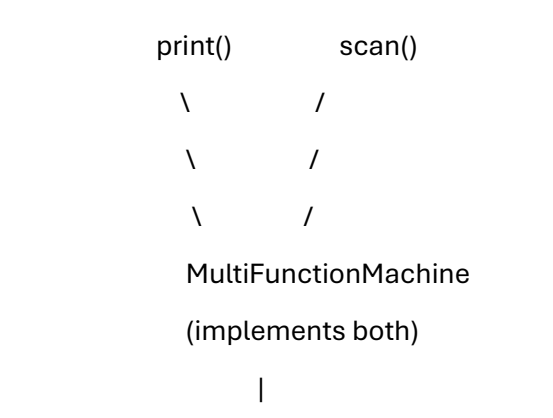
Machine Name: HP Printer

Machine ID: 101

Machine is printing

Machine is scanning

Hierarchy Diagram:-



Q18. Vehicle → Car → ElectricCar + Electric [Medium → Challenging]

Create Vehicle with start().

Create Car extends Vehicle with drive().

Create Electric interface with chargeBattery().

Create ElectricCar extends Car implements Electric.

Create an ElectricCar object and call start(), drive(), and chargeBattery().

Draw the structure. Concepts: multilevel inheritance + interface implementation + hybrid structure.

```
class Vehicle {
  void start() {
    System.out.println("Vehicle is starting");
  }
}

class Car extends Vehicle {
  void drive() {
    System.out.println("Car is driving");
  }
}

interface Electric {
  void chargeBattery();
}

class ElectricCar extends Car implements Electric {
  @Override
  public void chargeBattery() {
    System.out.println("Electric car battery is charging");
  }
}

public class Main {
  public static void main(String[] args) {
    ElectricCar e = new ElectricCar();

    e.start();
```

```
        e.drive();
        e.chargeBattery();
    }
}
```

OUTPUT:-

Vehicle is starting

Car is driving

Electric car battery is charging

Structure Diagram:-

```
        Vehicle
        start()
          |
          | extends
          |
         Car
        drive()
          |
          | extends
          |
      ElectricCar  -------- implements --------> Electric
          |                          chargeBattery()
          |
     start(), drive(),
     chargeBattery()
```

Q19. Employee → Developer + Programmer + Researcher [Challenging]

Create Employee with encapsulated name and employeeId and displayEmployee().

Create Programmer interface with writeCode().

Create Researcher interface with conductResearch().

Create Developer extends Employee implements Programmer, Researcher. Implement all required methods.

Draw the hierarchy. Concepts: encapsulation, inheritance, multiple interfaces, hybrid structure.

```
class Employee {
    private String name;
    private int employeeId;

    public void setName(String n) {
        name = n;
    }

    public String getName() {
        return name;
    }

    public void setEmployeeId(int id) {
        employeeId = id;
    }

    public int getEmployeeId() {
        return employeeId;
    }

    void displayEmployee() {
        System.out.println("Employee Name: " + name);
        System.out.println("Employee ID: " + employeeId);
    }
}

interface Programmer {
    void writeCode();
```

```
}

interface Researcher {
   void conductResearch();
}

class Developer extends Employee implements Programmer, Researcher {
   @Override
   public void writeCode() {
      System.out.println("Developer is writing code");
   }

   @Override
   public void conductResearch() {
      System.out.println("Developer is conducting research");
   }
}

public class Main {
   public static void main(String[] args) {
      Developer d = new Developer();

      d.setName("Rahul");
      d.setEmployeeId(101);

      d.displayEmployee();
      d.writeCode();
      d.conductResearch();
   }
}
```

OUTPUT:-

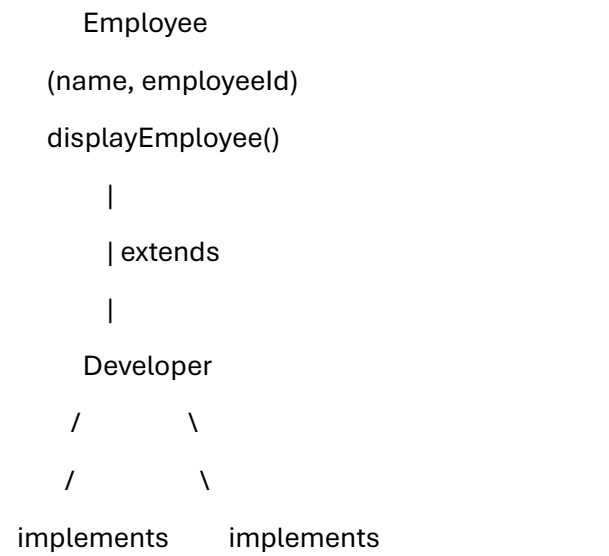
Employee Name: Rahul

Employee ID: 101

Developer is writing code

Developer is conducting research

Hierarchy Diagram:-



Q20. University Employee System — Integrated OOP [Challenging]

Design a university employee system.

Hierarchy:

Employee

/ \

Teacher Admin

| VisitingTeacher

Also create Researcher interface.

Employee:

• private employeeId, employeeName, salary

• getters and setters

• displayDetails()

• calculateSalary() Teacher:

• subject

• teach()

• implements Researcher

• override calculateSalary() VisitingTeacher:

• hoursWorked

• override calculateSalary() Admin:

• department • manageDepartment() Researcher:

• conductResearch() Your program must demonstrate:

• Encapsulation • Getter/setter • Single inheritance

• Multilevel inheritance • Hierarchical inheritance • Interface • Multiple inheritance through interfaces • Hybrid inheritance structure

• Method overriding • @Override • super where appropriateclass Employee {

```
  private int employeeId;
  private String employeeName;
  private double salary;

  public void setEmployeeId(int id) {
    employeeId = id;
  }

  public int getEmployeeId() {
    return employeeId;
  }

  public void setEmployeeName(String name) {
    employeeName = name;
  }
```

```
    public String getEmployeeName() {
        return employeeName;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public double getSalary() {
        return salary;
    }

    void displayDetails() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Employee Name: " + employeeName);
        System.out.println("Basic Salary: " + salary);
    }

    double calculateSalary() {
        return salary;
    }
}

interface Researcher {
    void conductResearch();
}

class Teacher extends Employee implements Researcher {
    String subject;
```

```
    void teach() {
        System.out.println("Subject: " + subject);
        System.out.println("Teacher is teaching");
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + 5000;
    }

    @Override
    public void conductResearch() {
        System.out.println("Teacher is conducting research");
    }
}

class VisitingTeacher extends Teacher {
    int hoursWorked;

    @Override
    double calculateSalary() {
        return hoursWorked * 500;
    }
}

class Admin extends Employee {
    String department;

    void manageDepartment() {
        System.out.println("Department: " + department);
```

```
        System.out.println("Admin is managing the department");
    }
}

public class Main {
    public static void main(String[] args) {

        Teacher t = new Teacher();
        t.setEmployeeId(101);
        t.setEmployeeName("Rahul");
        t.setSalary(30000);
        t.subject = "Java";

        System.out.println("----- Teacher Details -----");
        t.displayDetails();
        t.teach();
        t.conductResearch();
        System.out.println("Total Salary: " + t.calculateSalary());

        System.out.println();

        VisitingTeacher v = new VisitingTeacher();
        v.setEmployeeId(102);
        v.setEmployeeName("Priya");
        v.setSalary(0);
        v.subject = "Python";
        v.hoursWorked = 20;

        System.out.println("----- Visiting Teacher Details -----");
        v.displayDetails();
```

```
        v.teach();
        System.out.println("Hours Worked: " + v.hoursWorked);
        System.out.println("Total Salary: " + v.calculateSalary());

        System.out.println();

        Admin a = new Admin();
        a.setEmployeeId(103);
        a.setEmployeeName("Amit");
        a.setSalary(40000);
        a.department = "Administration";

        System.out.println("----- Admin Details -----");
        a.displayDetails();
        a.manageDepartment();
        System.out.println("Total Salary: " + a.calculateSalary());
    }
}
```

OUTPUT:-

```
----- Teacher Details -----
Employee ID: 101
Employee Name: Rahul
Basic Salary: 30000.0
Subject: Java
Teacher is teaching
Teacher is conducting research
Total Salary: 35000.0

----- Visiting Teacher Details -----
Employee ID: 102
```

Employee Name: Priya

Basic Salary: 0.0

Subject: Python

Teacher is teaching

Hours Worked: 20

Total Salary: 10000.0

----- Admin Details -----

Employee ID: 103

Employee Name: Amit

Basic Salary: 40000.0

Department: Administration

Admin is managing the department

Total Salary: 40000.0

Hierarchy Diagram:-

